Stubble Burning Early Warning

Satellite fires + wind = who gets smoke next, and a warning before it arrives

Hackathon track: 01 Air (track the air, warn the most exposed people, change what happens on bad days)
Build constraint: must use AWS open source tools and/or AWS cloud services
Document: problem analysis, competitor review, architecture, data needs, 11-day plan, demo, pitch, and a step-by-step plan for a coding agent

Assumptions to check. (1) The judging criteria file has not been received, so Section 14 uses the usual hackathon criteria. Replace it with the real list. (2) The plan assumes about 11 days are left. (3) Statistics, competitor details and API details are from general knowledge; verify them before quoting or coding against them. Section 16 lists what to verify.

1. One-paragraph summary

Every October and November, crop-residue fires across Punjab, Haryana and nearby states send smoke over the Indo-Gangetic plain, and Delhi-NCR's air regularly turns Poor to Severe. The fires are visible from satellites within hours, and the wind decides who gets the smoke, but most people only learn about it when the air is already bad. This system pulls fire hotspots (NASA FIRMS), wind and mixing-height forecasts, and live air-quality readings, estimates which districts, schools and hospitals will be downwind in the next 6 to 48 hours, and sends them a specific, local-language action alert (for example, move school sports indoors tomorrow morning). A Strands agent on Amazon Bedrock decides who needs which alert and writes it; a map dashboard shows officials the burning clusters and the likely smoke path.

Pitch line: "Satellites see the fire in hours. The smoke takes a day to arrive. We use that gap to warn people before they breathe it."

2. The problem: where it exists, who suffers, what they expect

2.1 Where it exists

- **Source region:** paddy-harvest areas of Punjab, Haryana and western Uttar Pradesh, mainly from about mid-October to mid-November, after the paddy harvest and before wheat sowing.
- **Affected region:** downwind areas of the Indo-Gangetic plain, most visibly Delhi-NCR, but also Chandigarh, Ludhiana, Amritsar, Lucknow and many towns in between.
- **Why it keeps happening:** the gap between harvest and the next sowing is short, burning is cheap and fast, and alternatives (happy seeder machines, in-situ decomposition, ex-situ biomass buyers) are not equally reachable for every farmer.

2.2 Who suffers and how

Group	How they suffer
City residents	Poor air for weeks: asthma and heart-lung flare-ups, eye and throat irritation, missed work, hospital visits. Most find out only after the smog arrives.
Children and schools	Outdoor sports and assemblies continue on bad days because schools have no early, local signal. Closures are announced late and broadly.
Hospitals and clinics	Surges in respiratory cases arrive with little notice, so staffing and stock are reactive.
Outdoor workers	Delivery riders, traffic police, construction and street vendors are exposed for hours every day.
Farmers	Many burn because of cost, time pressure and machinery access, then face fines and blame. They also breathe the smoke.
Authorities	Fire counts exist, but linking fires to who will be affected, and acting early, is manual and slow.

2.3 What each side expects from a solution

Who	Expects	Will reject it if
Residents and parents	Early, simple, local warning with a clear action	Alerts are generic, constant or in the wrong language
Schools and hospitals	Lead time of at least half a day and a specific recommendation	It only says AQI is bad, which they already know
District officials	Cluster-level view of where burning is concentrated and where smoke is heading	A noisy map with no priorities
Farmers	Support and alternatives, not just blame	It acts only as a surveillance and fine tool
Judges	Real data, working demo, measurable lead time, sensible AWS use	A pretty dashboard with no prediction or action

3. Our solution and its significance

3.1 The solution

- **Detect:** pull VIIRS and MODIS fire hotspots every few hours for Punjab, Haryana and neighbouring states. Filter to likely cropland fires.
- **Predict:** combine fire strength (fire radiative power), wind direction and speed, and mixing height to estimate a smoke exposure score for each 10 km grid cell, district, school and hospital over the next 6 to 48 hours.
- **Validate:** compare the score with observed PM2.5 and AQI at monitoring stations, and show the backtest (did upwind fires predict the later AQI rise, and by how many hours).
- **Act:** an agent chooses alert tier and audience, writes the message in the local language, and sends it. Schools get "move PE indoors", hospitals get a surge heads-up, residents get mask and activity guidance.
- **Support:** an optional farmer-side advisory points to alternatives (machine rental, subsidy schemes, residue buyers) instead of only reporting fires.
- **Show:** an official dashboard with burning clusters, smoke paths and alert history.

3.2 Why it matters (market and impact)

- Delhi-NCR alone has tens of millions of residents, and the Indo-Gangetic plain is among the most polluted regions in the world during this season.
- Stubble smoke is a seasonal but severe spike on top of year-round pollution, and its share of Delhi's PM2.5 varies a lot by day. Official source-apportionment tools report this share publicly.
- **Customers:** state and city governments and pollution control boards (B2G), school chains and hospital groups (B2B), and residents (freemium). Employers with outdoor workforces and event organisers are a further group.
- The same pipeline extends to any smoke source: forest fires, crop burning in other regions, or industrial plumes, so the technology is not limited to one season.

Cite sources for any number you put on a slide (CPCB, IITM-SAFAR or DSS, NASA FIRMS, a peer-reviewed study). One checked number beats five rounded guesses.

4. What others are doing, and where we win or lack

Existing effort	What it does	Gap	Us vs them
NASA FIRMS	Global near-real-time fire hotspot maps and downloads	Raw detections; no exposure, no alerts	We use it as our input and add prediction and action
CPCB Sameer app and AQI bulletins	Live AQI from monitoring stations, official advisories	Reports current air, not where smoke will go; generic advice	We warn ahead and by locality
IITM-SAFAR and the Delhi Decision Support System	AQI forecasts and source apportionment for Delhi-NCR	Strong science but city-level, mostly dashboards and bulletins, limited targeted push alerts	We add village to school-level targeting and direct alerts. They have far better physics and validation.
ICAR-IARI CREAMS and state remote-sensing centres (such as Punjab's)	Daily crop-residue fire counts and reports	Counting and monitoring, not exposure or personal alerts	We turn counts into who is affected next
IQAir, Plume-type apps, weather apps	Consumer AQI and short-term forecasts	No link to upwind fire sources; generic health tips	We explain the cause and give lead time
Government awareness and subsidy schemes	Machine subsidies, decomposer solutions, penalties	Not connected to real-time exposure	Our farmer advisory can point to them

Where we are weaker (say this honestly in Q and A)

- **Physics:** our plume model is simple. Operational systems use full chemistry-transport models. We position ours as a fast, transparent early-warning layer and show a backtest.
- **Satellite limits:** overpass timing can miss late-afternoon burning, and cloud or thick smoke can hide fires.
- **Data access:** official forecasts and some APIs are not freely available at the detail we would want.
- **Trust:** an unknown app has to earn trust; the fix is to cite sources, show validation, and partner with schools or an authority for a pilot.

Where we beat them

- Linking a specific upwind fire cluster to a specific downwind place and time, then telling that place what to do.
- Alerts for the people who need them most (schools, hospitals, outdoor workers), not just a public AQI number.
- Agentic automation: the system decides and acts every few hours without an analyst.
- Runs entirely on open data and serverless AWS, so it is cheap and easy to pilot in a district.

5. Technical look-over

5.1 Core modules

Module	Job	Notes
Fire ingestion	Fetch VIIRS and MODIS hotspots for the region, de-duplicate, tag state and district	Needs a free FIRMS key. Poll every 3 hours
Land-use filter	Keep fires on cropland, drop industrial and forest detections	ESA WorldCover or similar; simple raster lookup
Weather ingestion	Wind speed and direction, boundary layer height on a coarse grid for the next 48 h	Open-Meteo, a dozen points, interpolate
Exposure engine	Score each grid cell, district, school and hospital	Vectorised numpy. Section 5.2
AQI ingestion	Station PM2.5 and AQI for validation and for current-state context	OpenAQ v3 or CPCB open data
Backtest	Check that upwind fire exposure predicts later PM2.5 rises, and calibrate thresholds	Uses past-season FIRMS and AQI data
Alert agent	Pick tier and audience, write the message, send it	Strands agent with tools, Bedrock model
Notifier	Email, Telegram, WhatsApp sandbox	SMS only if India sender registration (DLT) is done
Dashboard API	Serve hotspots, exposure layers and alerts to the map	API Gateway + Lambda + DynamoDB

5.2 The exposure model (simple, explainable, tunable)

For every fire f and every target cell t, compute how much of that fire's smoke is likely to reach t in the next H hours, then sum over fires.

```
d       = distance(f, t)                    (km)
bearing = direction from f to t              (degrees)
wind_to = (wind_from_dir at f + 180) mod 360 (direction smoke travels)
dtheta  = angular difference(bearing, wind_to)
reach   = wind_speed_kmh * H                 (how far smoke can travel in H hours)

K_dir   = exp(-dtheta^2 / (2 * sigma^2))              sigma about 25 deg, widens with H
K_dist  = exp(-((d - 0.6*reach)^2) / (2 * (0.4*reach + 15)^2))   peaks near the travelled distance
K_mix   = 1 + a*(1 - min(BLH,1500)/1500) + b*(1 - min(ws,10)/10) low mixing height and weak wind trap smoke

contrib(f,t) = FRP_f * confidence_f * K_dir * K_dist * K_mix
exposure(t)  = sum over fires of contrib(f,t)
tier(t)      = Watch / Warning / Emergency by thresholds calibrated in the backtest
```

- Constants (sigma, a, b, thresholds) are placeholders. Calibrate them on last season's data so that tier changes line up with real PM2.5 rises.
- Add a baseline term so that already-high local AQI raises the tier, and use CPCB AQI bands (Good, Satisfactory, Moderate, Poor, Very Poor, Severe) in the messages.
- Upgrade path (future scope): replace the kernel with NOAA HYSPLIT trajectories or an ML model (XGBoost) trained on upwind fire features to predict station PM2.5.

5.3 Alert tiers

Tier	Meaning (tunable)	Typical action in the message
Watch	Smoke likely within 24 to 48 hours	Plan ahead: check masks and purifier filters, avoid scheduling outdoor events

Tier	Meaning (tunable)	Typical action in the message
Warning	Smoke likely within 6 to 24 hours or forecast AQI Poor to Very Poor	Schools: move sports indoors. Residents: limit outdoor exercise. Workers: reduce exposure hours
Emergency	Forecast AQI Severe or very strong upwind cluster	Schools: consider closing or going online. Hospitals: surge readiness. Everyone: stay indoors, wear N95 outdoors

5.4 Key technical risks and answers

Risk	Answer
Prediction accuracy	Show the backtest, call it an early-warning index not a precise forecast, keep the model transparent, tune thresholds with data
Satellite timing and cloud	State the limitation, use several satellites (VIIRS from more than one platform plus MODIS), and update every 3 hours
Alert fatigue	Tiers, per-user caps, only alert places whose tier changed, and quiet hours
Blaming farmers	Show clusters, not individuals; do not publish farm-level data; pair fire reports with support options
API limits and outages	Cache raw data in S3, keep a saved replay of a peak day for the demo
Cost	Serverless on free tier, run the heavy compute only every 3 hours, small Bedrock model, delete resources after the demo

6. Architecture

6.1 Client side

Client	Users	Key screens
Official dashboard (React PWA on Amplify)	District officials, pollution board, researchers	Map with fire hotspots, smoke exposure heat layer, predicted drift arrows, top burning clusters, alert log, backtest chart
Citizen web app and chat bot (Telegram or WhatsApp)	Residents and parents	Subscribe by location or pincode, choose language, receive alerts, see current exposure and advice
School and hospital console (simple web page)	Principals, hospital admins	Their site's exposure over the next 48 hours, recommended actions, one-tap acknowledge
Farmer advisory (stretch)	Farmers and local officers	Nearby alternatives and schemes, in the local language

6.2 Server side (all serverless)

```
EventBridge Scheduler (every 3 hours)
      |
Lambda: ingest  --> NASA FIRMS (fires)   Open-Meteo (wind, boundary layer)   OpenAQ / CPCB (AQI)
      |
Amazon S3 (raw JSON/CSV snapshots, replay files, backtest data)
      |
Lambda: exposure engine (numpy via AWS SDK for pandas layer)
      |
Amazon DynamoDB (hotspots, exposure by cell/district/site, sites, subscribers, alerts)
      |
Strands Agent (in Lambda) <--> Amazon Bedrock (decisions, local-language messages)
   tools: get_hotspots, get_exposure, get_sensitive_sites, check_recent_alerts,
          compose_alert, send_alert
      |
Notifier: SNS (email), Telegram Bot API, WhatsApp (Twilio sandbox)

API Gateway + Lambda (read APIs)  <--  Dashboard / apps (Cognito for officials)
Amazon Location Service (map)      CloudWatch (logs, traces)      AWS CDK (deploys everything)
```

6.3 How each AWS tool is used (plain language)

Tool	What it does here
Strands Agents (open source)	The agent brain. You write plain Python functions as tools; the agent decides which to call. It checks exposure, picks who to warn, avoids repeat alerts and drafts the message. Your open source eligibility point.
AWS CDK (open source)	Defines Lambdas, tables, schedules and the API in Python, deployed with one command. Second open source point.

Tool	What it does here
Powertools for AWS Lambda (open source)	Structured logs, tracing, and request validation for every Lambda. Third open source point.
Lambda	Runs each step: ingest, exposure, agent, API. No servers.
EventBridge Scheduler	Triggers the pipeline every 3 hours.
S3	Stores raw satellite and weather snapshots, backtest files and the demo replay.
DynamoDB	Fast storage for hotspots, exposure values, sites, subscribers and alerts.
Bedrock	Gives access to an LLM. Writes short alerts in Hindi, Punjabi or English and explains why an alert was sent.
SNS	Email alerts and admin notices.
API Gateway + Cognito	Secure API for the dashboard and logins for officials.
Amazon Location Service	Map and geocoding for the dashboard and pincode lookup.
Athena (optional)	SQL over S3 data for the backtest and analysis.

Amazon Pinpoint is being retired, so avoid it. Use SNS for email and Telegram or a Twilio WhatsApp sandbox for chat alerts. India SMS needs sender and template registration, so keep SMS out of the demo.

7. Tech stack

Layer	Choice	Why
Frontend	React (Vite) PWA, Tailwind, MapLibre with Amazon Location, or Leaflet	Fast to build, works on phones
Hosting	AWS Amplify Hosting or S3 + CloudFront	One-command deploy
Backend	Python 3.12 on Lambda, Powertools, Pydantic	One language for data, agent and API
Compute libs	numpy and pandas from the AWS SDK for pandas managed Lambda layer; shapely or geopandas only if needed	Avoids packaging pain
Database	DynamoDB on-demand, S3 for bulk files	No servers, free-tier friendly
Agent	Strands Agents SDK + Bedrock model	Open source orchestration
ML (optional)	XGBoost trained locally on past data, model file in S3	Avoids SageMaker setup time
IaC	AWS CDK (Python)	Reproducible deploy
Testing	pytest for the exposure engine and tiers, replay-based end-to-end test	Model bugs are the likeliest failure

8. Workflows

8.1 Client workflows

Actor	Flow
Resident or parent	Open the bot or web app, share location or pincode, choose language and alert level. Receive a tiered alert with the reason (for example, fires upwind, smoke expected by tomorrow morning) and an action. See current exposure on request.
School or hospital	Register the site and contact. Receive site-specific alerts with a recommended action. Acknowledge the alert so the dashboard shows who has seen it.
District official	Open the dashboard: top burning clusters, predicted smoke path, sites at risk, alert history. Export a daily summary.
Farmer (stretch)	Receive a seasonal reminder with nearby alternatives and schemes.

8.2 Developer workflow

- Build the exposure engine locally first, with saved FIRMS and weather files and pytest. No AWS needed.
- Run the backtest on last season's data and set initial thresholds.
- Wrap steps as Lambdas and deploy with cdk deploy to a personal dev stack.

- Use a replay mode: the pipeline can read a saved peak-day snapshot from S3 instead of live APIs, so the demo is repeatable.
- Keep keys in environment variables or Secrets Manager; never commit them.
- Freeze features on Day 9; spend the rest on demo, slides and rehearsal.

9. Implementation plan (about 11 days)

Day	Goal	Output
1	AWS account, budget alert, repo, CDK skeleton, FIRMS key, OpenAQ key. Download a saved peak-season FIRMS snapshot	Empty stack deploys; sample data in /data
2	Data clients: FIRMS, Open-Meteo, OpenAQ. District tagging for hotspots	Script prints fires per district for a day
3	Exposure engine (Section 5.2) with unit tests on synthetic cases	Fire due west of a school with westerly wind gives high score; opposite wind gives near zero
4	Backtest on last season: upwind exposure vs station PM2.5 lag. Initial thresholds	One chart and one summary number (correlation and typical lead time)
5	DynamoDB tables, ingest and exposure Lambdas, EventBridge schedule, S3 snapshots	Pipeline runs on schedule and writes results
6	Strands agent with tools and Bedrock messages; alert dedupe and caps	Agent trace producing tiered alerts for seeded sites
7	Notifier: email, Telegram bot, optional WhatsApp sandbox. Subscriber registration	Test alert arrives on a real phone in Hindi
8	Official dashboard: map, hotspots, exposure layer, drift arrows, alert log, backtest chart	Full map demo against replay data
9	School console and impact counters. Feature freeze. One stretch item only	Acknowledge flow; people and sites covered
10	Deploy clean stack, bug fixes, backup demo video, slides	Video and deck ready
11	Buffer, rehearsal, Q and A prep	Two timed run-throughs

If you fall behind, cut in this order: farmer advisory, WhatsApp, ML model (keep the kernel), land-use filter, Athena. Never cut the backtest chart, the agent trace, or the replay mode.

10. Innovative angles and helpful APIs

10.1 Ideas that add impact without much work

- **Backtest as proof:** a single chart showing fire exposure rising before AQI rises is the most persuasive slide you can have.
- **Lead-time headline:** report "warned X hours before the AQI peak" for the backtest days.
- **School-first targeting:** use OpenStreetMap school and hospital locations so the demo shows named places, not just a heat map.
- **Action cards:** each alert includes one concrete action by audience (school, hospital, rider, parent).
- **Local-language alerts:** Bedrock writes Hindi and Punjabi versions cheaply; a strong demo moment.
- **Cluster view, not farm view:** grid-level fire clusters protect individual farmers and are still useful for authorities.
- **Support-linked advisory:** pair fire reports with machine rental and scheme information so the tool helps the cause, not only the symptom.
- **Replay mode:** run a recorded peak day at 10x speed during the demo to show the whole season story in one minute.

10.2 Free or cheap APIs and datasets

Source	Use
NASA FIRMS API (VIIRS S-NPP, NOAA-20 and MODIS near-real-time, plus archive)	Fire hotspots with location, time, FRP and confidence. Free MAP_KEY. Check the area API format and day-range limit in the docs
Open-Meteo forecast API	Wind speed, wind direction, boundary layer height, temperature. Free, no key; confirm variable names in the docs
OpenAQ v3 API	Station PM2.5 and other pollutants for validation. Free key
CPCB open data and data.gov.in	Official AQI station data where available
OpenStreetMap Overpass API	Schools, hospitals and clinics locations
Admin boundaries (GADM or Datameet)	District and state polygons for tagging
ESA WorldCover or ISRO Bhuvan land cover	Filter fires to cropland
WorldPop	Population per grid cell for people-exposed estimates
NOAA HYSPLIT (stretch)	Proper trajectory model for the future-scope slide
Telegram Bot API and Twilio WhatsApp sandbox	Free alert delivery for the demo

11. Data required, in detail

Dataset	Key fields	Source / how to get it
Fire hotspots	lat, lon, acq_date, acq_time, satellite, FRP, confidence, brightness, day/night	FIRMS API; last-season archive for the backtest
Weather forecast and reanalysis	wind_speed, wind_direction, boundary_layer_height, temperature, hourly for 48 h; past hourly for backtest	Open-Meteo forecast and historical APIs
Air quality observations	station_id, lat, lon, timestamp, PM2.5, PM10, AQI	OpenAQ v3, CPCB data
Admin boundaries	state, district polygons	GADM or Datameet
Land cover	cropland mask raster	ESA WorldCover or Bhuvan
Population	people per grid cell	WorldPop
Sensitive sites	site_id, type (school, hospital, clinic), name, lat, lon, district, contact	OpenStreetMap; plus a few seeded demo sites with fake contacts
Subscribers	user_id, lat or pincode, language, channel, contact, role, opt_in, alerts_today	Self-registration; use test accounts for the demo
Alerts	alert_id, area_id, tier, reason, message, language, recipients, created_at, acknowledged	Generated by the agent
Backtest results	date, area, predicted tier, observed AQI change, lead time hours	Computed offline; stored in S3
Demo replay file	one saved peak day of fires, wind and AQI	Captured from last season or from a live peak day

The season runs roughly mid-October to mid-November, so live data should be rich during your hackathon window. Save a few days of snapshots for replay anyway; live APIs and cloud cover can both let you down on demo day.

Privacy basics

- Opt-in only, coarse location (pincode or neighbourhood), one-tap unsubscribe, no sale of data.
- Fire data is aggregated to clusters in public views. No individual farm or farmer is named.
- No real phone numbers or personal data in the repo, slides or demo video.

12. Ecosystem, integration and pitch

12.1 The ecosystem

- **Data layer:** satellites (FIRMS), weather models, CPCB and open sensor networks feed one pipeline.
- **Authorities:** district administration and pollution boards use the dashboard for targeting field teams and for public advisories.

- **Schools and hospitals:** receive site-specific alerts; school chains and hospital groups can integrate alerts into their own systems through the API.
- **Citizens:** chat-based alerts first (no app install), PWA second.
- **Employers and platforms:** delivery platforms, logistics and construction firms can use the API to adjust outdoor work hours on bad days.
- **Farmer support:** link to machine rental, subsidy information and residue buyers, so reducing burning is part of the loop.
- **Revenue model:** free for citizens, paid API or dashboard subscriptions for institutions and companies, and government pilot contracts.

12.2 Pitch structure (3 to 5 minutes)

Beat	Say
Hook (20 s)	Satellites spot the fire in hours. The smoke takes a day to reach a city. We use that gap.
Problem (30 s)	Every autumn, millions breathe smoke they could have been warned about. One sourced statistic.
Solution (30 s)	Fires, wind, mixing height give a smoke exposure score; an agent warns schools, hospitals and residents with a clear action.
Live demo (90 s)	See Section 13.
Proof (20 s)	Backtest chart: exposure rose X hours before AQI on real days.
Why AWS and open source (20 s)	Strands agent, CDK, Powertools, Lambda, DynamoDB, Bedrock, EventBridge. Serverless and cheap to pilot.
Impact and ask (20 s)	People and sites covered, hours of lead time. Pilot with one district's schools. Repeat the pitch line.

13. Demo plan

Step	What the audience sees	What you say
1	Map of Punjab-Haryana-Delhi with today's fire hotspots and an AQI layer	"These are the fires detected in the last few hours."
2	Click Run pipeline. Agent trace shows tools being called (hotspots, exposure, sites)	"The agent pulls fires and wind, then works out who is downwind."
3	Smoke exposure heat layer and drift arrows appear toward a city; named schools turn amber and red	"Same fires, different wind, different victims."
4	Switch the wind in the replay to another direction and the exposure moves	"It reacts to the wind, not just to distance."
5	Phone receives a Hindi (or Punjabi) alert for a school with a clear action	"The principal gets a specific action, half a day early."
6	Backtest chart: exposure rising before the AQI rise at a station	"We checked this on real days from last season."
7	Alert log and acknowledgements; people and sites covered counter	"Officials see who was warned and who acknowledged."
8	Architecture slide	"Serverless on AWS, orchestrated by an open source Strands agent."

- Pre-record a 2 minute backup video of the full flow.
- Use the replay file so the result is identical every time; show one live API call as proof it also works with fresh data.
- Prepare for questions on accuracy (backtest and honest limits), satellite blind spots, farmer fairness, and who pays.

14. Judging criteria mapping (assumed criteria)

These are typical hackathon criteria, since the real list was not received. Swap in the actual criteria and keep the approach of one line of proof per criterion.

Criterion	How this scores	Proof to show
Relevance to track (Air)	Tracks air, warns the most exposed, changes what happens on bad days, which mirrors the track brief	Alert examples for schools and hospitals
Innovation	Links upwind fires to downwind places and acts with an agent, instead of showing an AQI dashboard	Agent trace, drift map
Technical depth	Data pipeline, exposure model, backtest, agent tools, serverless architecture	Backtest chart, unit tests, architecture slide
Use of AWS and open source	Strands, CDK, Powertools plus Lambda, DynamoDB, S3, EventBridge, Bedrock, SNS	One slide mapping each tool to its job
Feasibility and scalability	Open data, serverless, region-agnostic pipeline	Live working demo, cost note
Impact	Lead time before smoke arrives, people and sites covered	Counters and backtest summary
Presentation and demo	Tight story, repeatable live flow, honest limits	Rehearsed script and backup video

15. Implementation plan for a coding agent

Paste the **Master prompt** first, then give the steps one at a time. Do not move on until the 'Done when' check passes.

Master prompt

```
You are building a Stubble Burning Early Warning system on AWS.
It ingests NASA FIRMS fire hotspots, Open-Meteo wind and boundary-layer forecasts and OpenAQ station data for
Punjab, Haryana and Delhi-NCR, computes a smoke exposure score per grid cell, district and sensitive site
(schools, hospitals), and sends tiered local-language alerts through a Strands agent.
REQUIRED AWS STACK (hackathon rule: must use AWS open source tools and/or AWS services; we use both):
- Open source: Strands Agents SDK, AWS CDK (Python), Powertools for AWS Lambda.
- Services: Lambda, API Gateway, EventBridge Scheduler, S3, DynamoDB, Bedrock, SNS, Cognito,
  Amazon Location Service. Athena is optional. Do NOT use Amazon Pinpoint (being retired).
- Region: choose one where the Bedrock model we use is available, and confirm model access in the
  console before writing the agent. Keep the model id and region in environment variables.
- Pin library versions in requirements files and note them in the README.
Rules:
- Python 3.12, AWS CDK (Python), AWS Lambda Powertools, Pydantic, pytest, numpy. Frontend: React + Vite + MapLibre.
- The exposure engine, tiers and dedupe logic are pure Python with no AWS imports and fully unit tested.
- Use the Strands Agents SDK. Tools must be small, typed functions. The agent may choose audience and write
  messages, but tier thresholds come from config, not from the LLM.
- Support a REPLAY mode: read saved snapshots from S3 or /data instead of live APIs.
- Never publish farm-level data; aggregate fires to grid clusters in public views.
- No secrets in code; read keys from environment variables or Secrets Manager.
- Verify API parameters against official docs before coding clients; make base URLs configurable.
- After each step: run tests, summarise changes and assumptions, then stop and wait.
Repo layout: /infra (CDK), /backend (lambdas, core, agent), /frontend, /scripts, /data, /tests.
```

Steps

#	Task for the agent	Done when
1	Scaffold repo and CDK app with empty stacks for data, ingest, agent and API. Makefile targets: test, deploy, replay.	cdk synth runs; pytest runs one sample test
2	Write clients/firms.py for the FIRMS area CSV API for a Punjab-Haryana-NCR bounding box (configurable), with parsing to a typed model. Cache raw responses to S3 or /data.	Run for one past day; hotspot count printed; saved to /data
3	Write clients/weather.py for Open-Meteo hourly wind speed, direction and boundary layer height at a coarse grid of points, plus interpolation to any lat/lon and hour.	Interpolated wind at a test point matches a direct query within tolerance
4	Write clients/aqi.py for OpenAQ v3 stations in the region, returning PM2.5 and timestamps. Handle missing data.	Station time series for one week saved
5	Add geo utilities: district tagging with polygons, haversine, bearing, grid builder (0.1 degree), cropland filter hook.	Unit tests on known coordinates pass
6	Implement core/exposure.py per Section 5.2, vectorised with numpy. Config for sigma, a, b, horizons and thresholds. Return per-cell score, tier and top contributing clusters.	Tests: fire upwind gives high score; fire downwind gives near zero; calm wind and low mixing height raise score; runtime for 2000 fires and 3000 cells under 3 seconds

#	Task for the agent	Done when
7	Write scripts/backtest.py: for last season's days, compute exposure at stations and compare to later PM2.5 rises at lags 0 to 48 h. Output a chart, correlation per lag and suggested thresholds.	Chart and summary file saved; thresholds written to config
8	Add DynamoDB tables in CDK (hotspots, exposure, sites, subscribers, alerts) and repository classes. Load sites from OpenStreetMap Overpass for the region, plus 10 seeded demo sites.	Seed command fills tables; counts printed
9	Build Lambdas with Powertools: ingest (fire, weather, AQI to S3), exposure (reads S3, writes DynamoDB), scheduled by EventBridge every 3 hours. Replay flag supported.	Manual invoke in replay mode writes exposure rows
10	Implement the Strands agent with tools get_hotspots, get_exposure, get_sensitive_sites, check_recent_alerts, compose_alert (language aware), send_alert. Dedupe rule: alert only when a site's tier increases or 12 hours pass. Template fallback if Bedrock fails.	Agent run in replay mode produces alerts for seeded sites; trace logged
11	Notifier: SNS email, Telegram bot with /start to subscribe and language choice, optional WhatsApp via Twilio sandbox, dry-run flag, per-user daily cap.	Real Telegram message in Hindi received; dry-run logs only
12	API Gateway + Lambda read endpoints: /hotspots, /exposure, /sites, /alerts, /backtest. Cognito authorizer for official routes.	curl smoke script passes on deployed stack
13	Frontend: map with hotspot clusters, exposure heat layer, drift arrows, site markers coloured by tier, alert log, backtest chart, replay speed control. Simple school console with acknowledge button.	Full demo flow works in browser against replay data
14	Impact counters: people covered (WorldPop or population proxy), sites alerted, acknowledgements, average lead time from backtest.	Counters match seeded scenario
15	scripts/demo.py: reset, load replay day, run pipeline, send test alerts. Add an end-to-end test and a README with deploy, replay and demo steps.	One command reproduces the demo state; new teammate can deploy from README

16. Risks, honest limits, and what to verify

- **Verify API details:** FIRMS area endpoint format, key and day-range limits; Open-Meteo variable names (including boundary layer height); OpenAQ v3 endpoints and keys. Read the docs before coding the clients.
- **Verify numbers before quoting:** Delhi-NCR population, stubble share of PM2.5, AQI band definitions, season dates, and any health claims. Cite CPCB, IITM or peer-reviewed sources.
- **Model claims:** call it an early-warning index. Show the backtest and its limits. Do not claim exact PM2.5 forecasts.
- **Satellite limits:** overpass timing and cloud or smoke can hide fires; say so before a judge does.
- **Messaging compliance:** India SMS needs DLT registration and WhatsApp business messaging needs verification. Use Telegram, email and the Twilio sandbox for the demo.
- **Ethics:** keep farmer data aggregated; pair reporting with support options.
- **Bedrock access:** enable model access early and keep a template-message fallback.
- **Replace assumptions:** add the real judging criteria and deadline, then adjust Sections 9 and 14.